

# Java Bushidô

Do Java 8 ao 25: cortes precisos na evolução



Luis Zancanela

# Sumário

## Java Bushidō — Do Java 8 ao 25

Este ebook é um guia prático e acessível sobre a evolução do Java, pensado especialmente para quem está começando na carreira de desenvolvimento. Cada capítulo acompanha uma versão importante da linguagem, destacando os recursos que mais impactam o dia a dia de quem programa.

[Prefácio: O Código como Caminho](#)

[Arco I: Antes da Katana](#)

[Arco II: O Corte Inicial \(Java 8\)](#)

[Arco III: Disciplina e Leveza \(Java 11\)](#)

[Arco IV: O Código do Mestre \(Java 17\)](#)

[Arco V: A Fluidez do Aço \(Java 21\)](#)

[Arco VI: A Lamina Afiada \(Java 25\)](#)

[Arco Final: O Caminho do Java Bushidō](#)

[Epílogo: Honra no Código](#)

[Códigos de exemplo no GitHub](#)

Prefácio

# **PREFÁCIO**

## **O CÓDIGO COMO CAMINHO**



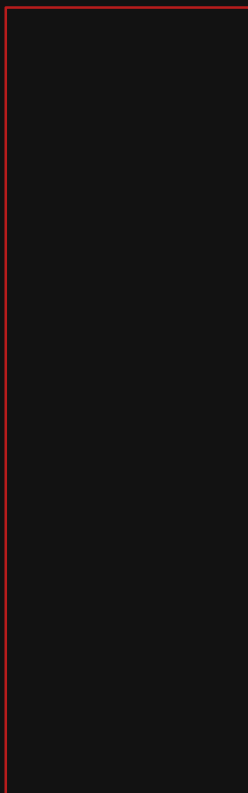
# O Código como Caminho

## O caminho do código

Programar não é apenas escrever instruções para uma máquina. É um processo contínuo de aprendizado, em que cada linha de código reflete uma decisão, uma intenção e um nível de maturidade técnica. Assim como um samurai que treina diariamente, lapidando sua técnica e sua postura, o desenvolvedor evolui junto com as tecnologias que domina.

Java é um exemplo notável dessa jornada. Desde sua criação, a linguagem seguiu um princípio que poucos ecossistemas conseguem manter: evoluir sem quebrar o que já funciona. Isso significa que o código que você escreveu ontem continua válido amanhã, mesmo com as novidades que cada versão traz (salvo ajustes e depreciações naturais da evolução).

Disciplina técnica é o que separa um código que apenas funciona de outro que é confiável, legível e fácil de manter. Não se trata de seguir regras rígidas, mas de construir hábitos que tornam o desenvolvimento mais previsível e colaborativo.



# ANTES DA KATANA



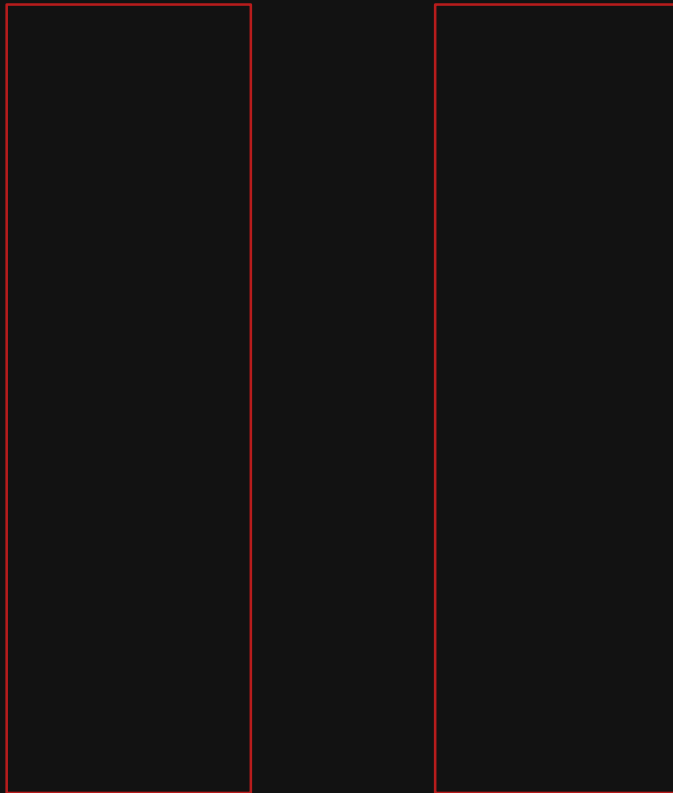
# O Java Antes da Versão 8

## Arco I — Antes da Katana

Antes do Java 8, a linguagem era predominantemente imperativa. Isso significa que o programador precisava detalhar cada passo: criar variáveis auxiliares, iterar com loops tradicionais e escrever blocos de código extensos para tarefas relativamente simples. Não era raro encontrar classes extensas para representar objetos simples.

Esse estilo verboso tinha suas vantagens, pois era explícito e previsível, mas também gerava repetição. Padrões como classes anônimas internas para tratar eventos ou *callbacks* tornavam o código denso e difícil de ler, especialmente para quem estava aprendendo.

A programação funcional, tão presente em linguagens como Python e JavaScript, praticamente não existia no Java dessa época. Não havia *lambdas*, não havia *streams*, e manipular coleções exigia loops manuais. A evolução era lenta, e essa estabilidade acabou se tornando sua maior virtude nos ambientes corporativos.



# O CORTE INICIAL

## JAVA 8



# Java 8 | Visão Geral

## Arco II — O Corte Inicial

O Java 8, lançado em março de 2014, foi a maior revolução da linguagem. Pela primeira vez, o Java ganhou suporte a programação funcional com a introdução de expressões *lambda* e a *Stream API*. Essas mudanças transformaram a forma como os desenvolvedores escrevem código, permitindo soluções mais concisas e expressivas.

Além das *lambdas* e *streams*, o Java 8 trouxe uma *API* de data e hora completamente nova (*java.time*), interfaces com métodos *default*, e o tipo *Optional* para lidar com valores nulos de forma mais segura. Juntas, essas features estabeleceram a base do Java moderno.

Para quem está começando, dominar o Java 8 é essencial, quase todo projeto Java em produção hoje utiliza pelo menos *lambdas*, *streams* e *java.time*. Nos próximos três slides, vamos explorar cada uma das três features citadas desta versão.



# Java 8 | Lambdas

## O Golpe Sem Forma

Lambdas são funções anônimas que simplificam o código ao eliminar classes anônimas verbosas. Elas permitem passar comportamento como parâmetro, tornando *APIs* como *Collections* e *Streams* muito mais expressivas. São a base de todo o Java moderno.

```
Lambdas.java

public class Lambdas {

    public static void main(String[] args) {
        List<String> samurais = Arrays
            .asList("Musashi",
                "Yoshi",
                "Hattori");

        // Iterando sobre samurais utilizando lambda
        samurais.stream()
            .forEach(nome ->
                System.out.println(
                    "🗡️ " +
                    nome +
                    " está em treino"
                ));

        // Usando como callback para execução de técnicas
        Runnable tecnicaDeCorte = () ->
            System.out.println("⚡ Iaijutsu executado!");
        // Executando a técnica
        tecnicaDeCorte.run();
    }
}
```

```
🗡️ Musashi está em treino
🗡️ Yoshi está em treino
🗡️ Hattori está em treino
⚡ Iaijutsu executado!
```

# Java 8 | Stream API

## A Lâmina em Fluxo

Streams permitem processar coleções de forma declarativa com operações como *filter* e *map*. Em vez de loops manuais, você descreve o que quer e o Java cuida da execução, inclusive em paralelo com *parallelStream()*.

```
Streams.java

public class Streams {

    static class Samurai {
        private String nome;
        private int nivel;
        private String escola;

        Samurai(String nome, int nivel, String escola) {
            this.nome = nome;
            this.nivel = nivel;
            this.escola = escola;
        }

        public String getNome() { return nome; }
        public int getNivel() { return nivel; }
    }

    public static void main(String[] args) {
        List<Samurai> guerreiros = Arrays.asList(
            new Samurai("Musashi", 10, "Niten'ichi"),
            new Samurai("Yoshi", 7, "Tenshin"),
            new Samurai("Hattori", 9, "Iga")
        );

        // Mestres: os que atingiram nível 8+
        List<String> mestres = guerreiros.stream()
            .filter(s -> s.getNivel() >= 8)
            .map(s -> s.getNome())
            .sorted()
            .collect(Collectors.toList());

        System.out.println("✂ Mestres de nível 8+:");
        mestres.forEach(mestre -> System.out.println("    • " + mestre));
    }
}
```

```
✂ Mestres de nível 8+:
• Hattori
• Musashi
```

# Java 8 | API de Data e Hora

## O Tempo do Dojō

O pacote *java.time* substituiu a antiga API de datas (*Date/Calendar*) com classes imutáveis e intuitivas como *LocalDate*, *LocalTime* e *ZonedDateTime*. Operações com datas ficaram simples e seguras, sem surpresas com fusos horários.

```
JavaTime.java

public class JavaTime {

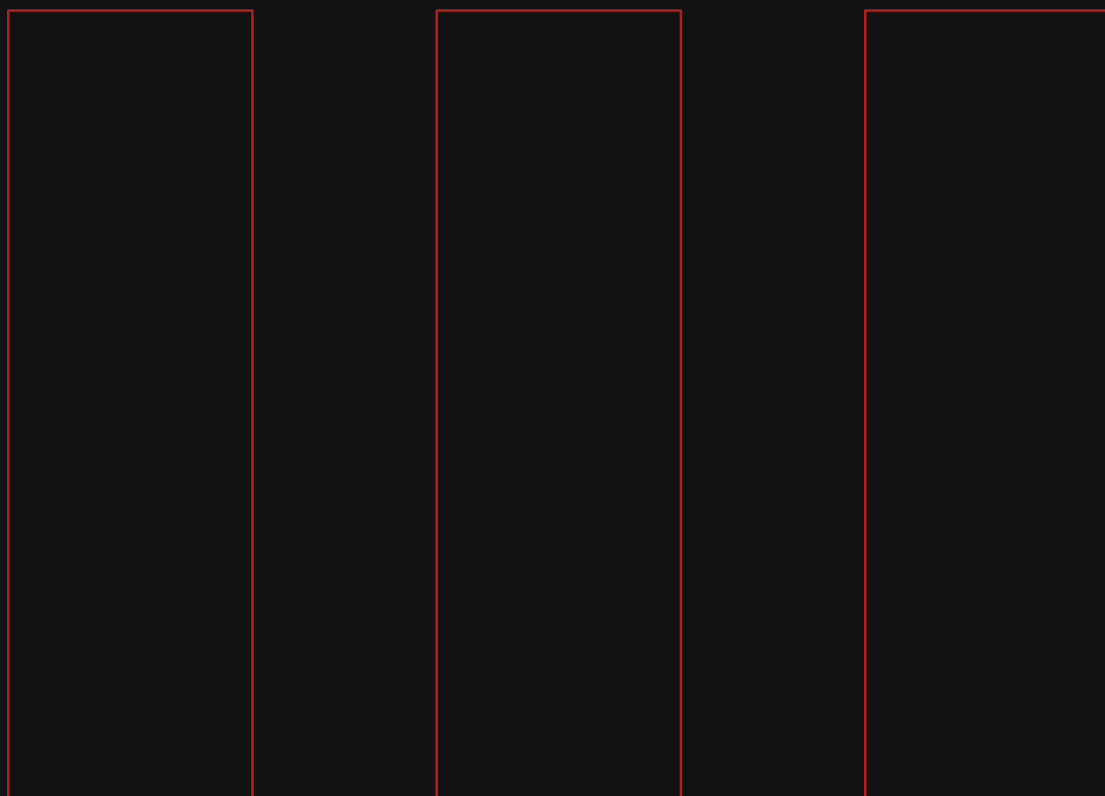
    public static void main(String[] args) {
        // Data de hoje no treino do samurai
        LocalDate hoje = LocalDate.now();
        LocalDate inicioTreino = hoje;
        LocalDate fimTreino = inicioTreino.plusDays(90);

        // Dias até a próxima promoção
        LocalDate dataProvaDePromocao = LocalDate.of(2026, 5, 31);
        long diasParaPromocao = ChronoUnit.DAYS.between(hoje,
dataProvaDePromocao);

        System.out.println("📅 Hoje: " + hoje);
        System.out.println("📅 Início do treino: " + inicioTreino);
        System.out.println("📅 Fim do treino em 90 dias: " + fimTreino);
        System.out.println("⌚ Dias até a promoção: " + diasParaPromocao);

        // Horário preciso com fuso do Japão
        ZonedDateTime treinoNoJapao =
ZonedDateTime.now(ZoneId.of("Asia/Tokyo"));
        System.out.println("🌅 Treino em Tokyo: " + treinoNoJapao);
    }
}
```

```
📅 Hoje: 2026-05-11
📅 Início do treino: 2026-05-11
📅 Fim do treino em 90 dias: 2026-08-09
⌚ Dias até a promoção: 20
🌅 Treino em Tokyo: 2026-05-11T23:31:42.249418400+09:00[Asia/Tokyo]
```



# **DISCIPLINA E LEVEZA JAVA 11**



# Java 11 | Visão Geral

## Arco III — Disciplina e Leveza

O Java 11, lançado em setembro de 2018, foi a primeira versão *LTS* (*Long-Term Support*) após o Java 8. Essa versão marcou a transição para o novo ciclo de releases semestrais e consolidou melhorias que tornaram o dia a dia do desenvolvedor mais leve e produtivo.

Entre as novidades mais relevantes estão o *HttpClient* nativo (que elimina a necessidade de bibliotecas externas para chamadas HTTP), melhorias importantes na classe *String* e a possibilidade de executar arquivos *.java* diretamente com comando **java** sem a etapa de compilar. São mudanças que parecem pequenas isoladamente, mas juntas reduzem significativamente a quantidade de código auxiliar em qualquer projeto.

O Java 11 é a versão mínima recomendada para novos projetos hoje. Se você ainda está em uma base Java 8, planejar a migração para o 11 é o primeiro passo concreto rumo ao Java moderno.

# Java 11 | HttpClient

## O Caminho do Mensageiro

O *HttpClient* é o cliente HTTP nativo do Java. Suporta HTTP/2, chamadas síncronas e assíncronas, e elimina a necessidade de bibliotecas externas como *Apache HttpClient* para chamadas REST simples.

```
HttpClientExample.java

public class HttpClientExample {

    private static final Pattern QUOTE_PATTERN =
        Pattern.compile("\"q\"\\s*:\\s*\"([^\"]*)\"");
    private static final Pattern AUTHOR_PATTERN =
        Pattern.compile("\"a\"\\s*:\\s*\"([^\"]*)\"");

    public static void main(String[] args) {
        // O dojô consulta uma fonte pública de conhecimento
        HttpClient client = HttpClient.newHttpClient();
        try {
            HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("https://zenquotes.io/api/random"))
                .header("Accept", "application/json")
                .build();
            HttpResponse<String> response =
                client.send(request, HttpResponse.BodyHandlers.ofString());
            String json = response.body();
            String frase = extrairCampo(json, QUOTE_PATTERN);
            String autor = extrairCampo(json, AUTHOR_PATTERN);

            System.out.println("📖 Conhecimento recebido: \"" +
                frase +
                "\" - " +
                autor
            );
        } catch (IOException | InterruptedException e) {
            Thread.currentThread().interrupt();
            System.out.println(
                "⚠️ Não foi possível consultar o conhecimento " +
                e.getMessage()
            );
        }
    }

    private static String extrairCampo(String json, Pattern pattern) {
        Matcher matcher = pattern.matcher(json);
        return matcher.find() ? matcher.group(1) : "desconhecido";
    }
}
```

```
📖 Conhecimento recebido: "Work harder on yourself than you do on your job." - Jim Rohn
```

# Java 11 | Melhorias em String

## A Arte da Palavra Precisa

Novos métodos como *isBlank()*, *strip()*, *lines()* e *repeat()* simplificam operações comuns com *strings*. São pequenas adições que eliminam dezenas de funções utilitárias espalhadas pelo projeto.

StringsUpdate.java

```
public class StringsUpdate {  
  
    public static void main(String[] args) {  
        // Poema do Bushidô com múltiplas linhas  
        String poema = "  Corte sem esforço\nMente como água\nAço afiado  ";  
  
        // Validar se um juramento não está vazio ou contém apenas espaços  
        String juramento = "Honra ao dojo";  
        if (!juramento.isBlank()) {  
            System.out.println("✓ Juramento válido");  
        }  
  
        System.out.println();  
        // Processar cada linha de sabedoria do samurai  
        System.out.println("📖 Ensinaamentos do Bushidô:");  
        poema.lines()  
            .forEach(linha -> System.out.println("  " + linha.trim()));  
  
        System.out.println();  
        // O mesmo ataque é praticado 3 vezes para dominar  
        String ataque = "🗡️";  
        System.out.println("🔪 Treino de repetição:");  
        System.out.println(ataque.repeat(3));  
    }  
}
```

```
✓ Juramento válido  
  
📖 Ensinaamentos do Bushidô:  
  Corte sem esforço  
  Mente como água  
  Aço afiado  
  
🔪 Treino de repetição:  
🗡️🗡️🗡️
```



# Java 11 | Execução direta de arquivos .java

## A Precisão de um Único Golpe

O Java 11 trouxe uma melhoria simples, mas extremamente prática para o dia a dia: agora é possível executar um arquivo .java diretamente, sem precisar chamar o *javac* manualmente. Essa funcionalidade aproxima o Java de linguagens usadas para scripts rápidos, como Python, e facilita experimentação, prototipação e ensino.

```
CompilaExecuta.java

public class CompilaExecuta {

    public static void main(String[] args) {
        // Na raiz de JavaVersions, execute sem chamar javac antes:

        // CMD:
        // java
        .\dev\zancanela\javaversions\java11\compilaexecuta\CompilaExecuta.java

        // PowerShell:
        // java
        .\dev\zancanela\javaversions\java11\compilaexecuta\CompilaExecuta.java

        // bash:
        // java
        ./dev/zancanela/javaversions/java11/compilaexecuta/CompilaExecuta.java

        System.out.println("Dojo iniciado sem compilacao manual.");
        System.out.println("Java 11 permite executar o .java direto.");
        System.out.println("Samurai em treino: foco na pratica rapida.");
    }
}
```

```
PS JavaVersions> java .\dev\zancanela\javaversions\java11\compilaexecuta\CompilaExecuta.java
Dojo iniciado sem compilacao manual.
Java 11 permite executar o .java direto.
Samurai em treino: foco na pratica rapida.
```





# **O CÓDIGO DO MESTRE JAVA 17**



# Java 17 | Visão Geral

## Arco IV — O Código do Mestre

O Java 17, lançado em setembro de 2021, é a segunda versão LTS após o Java 8 e trouxe recursos que mudam fundamentalmente a forma de modelar dados e hierarquias em Java. É a versão onde o Java começou a parecer uma linguagem verdadeiramente moderna.

Os três pilares desta versão são *Records* (classes de dados imutáveis com uma única linha), *Pattern Matching* para *instanceof* (que elimina *casts* manuais) e *Sealed Classes* (que permitem controlar quem pode estender uma classe). Juntos, esses recursos formam a base para um design de domínio mais robusto e expressivo.

Para equipes que já migraram do Java 8 para o 11, o salto para o 17 e o próximo passo natural. Os ganhos em legibilidade e segurança do código são imediatos e significativos.

# Java 17 | Records

## O Registro Imutável

*Records* criam classes de dados imutáveis em uma única linha. O compilador gera automaticamente construtor, *getters*, *equals*, *hashCode* e *toString*. Ideais para *DTOs*, respostas de API e qualquer container de dados.

```
Records.java

public class Records {

    // Um Samurai é imutável: seus atributos essenciais não mudam
    public record Samurai(String nome, int nivel, String escola, LocalDate dataEntrada) {
        public Samurai { // construtor compacto com validação
            if (nivel < 0 || nivel > 10) {
                throw new IllegalArgumentException("Nível deve estar entre 0 e 10");
            }
        }
        public int anosDeServico() {
            return (int) ChronoUnit.YEARS.between(dataEntrada, LocalDate.now());
        }
    }

    public static void main(String[] args) {
        Samurai musashi =
            new Samurai("Musashi", 10, "Niten'ichi",
                LocalDate.of(2010, 1, 1)
            );
        Samurai yoshi =
            new Samurai("Yoshi", 7, "Tenshin",
                LocalDate.of(2018, 6, 15)
            );

        System.out.println("✂ Samurais (record - imutáveis):");
        List.of(musashi, yoshi).forEach(System.out::println);

        System.out.println();
        System.out.println("💡 Records: concisos, imutáveis e com validação no construtor compacto.");
    }
}
```

```
✂ Samurais (record - imutáveis):
Samurai[nome=Musashi, nivel=10, escola=Niten'ichi, dataEntrada=2010-01-01]
Samurai[nome=Yoshi, nivel=7, escola=Tenshin, dataEntrada=2018-06-15]

💡 Records: concisos, imutáveis e com validação no construtor compacto.
```

# Java 17 | Pattern Matching (instanceof)

## O Olhar do Mestre

*Pattern matching* funde a verificação de tipo e o *cast* em uma única expressão: `if (obj instanceof String s)` já cria a variável tipada. Elimina *casts* manuais e reduz o risco de *ClassCastException*.

```
PatternMatchingInstanceOf.java

public class PatternMatchingInstanceOf {

    sealed interface Guerreiro permits SamuraiAtaque, SamuraiDefesa {}
    record SamuraiAtaque(String nome, int dano) implements Guerreiro {}
    record SamuraiDefesa(String nome, int armadura) implements Guerreiro {}

    public static void main(String[] args) {
        // Diferentes tipos de guerreiros, tratados com instanceof pattern matching
        Guerreiro combatente = new SamuraiAtaque("Hattori", 8);

        if (combatente instanceof SamuraiAtaque samurai) {
            System.out.println("🗡️ " + samurai.nome() +
                               " ataca com poder " + samurai.dano());
        } else if (combatente instanceof SamuraiDefesa samurai) {
            System.out.println("🛡️ " + samurai.nome() +
                               " defende com armadura " + samurai.armadura());
        } else {
            System.out.println("👤 Guerreiro desconhecido.");
        }

        System.out.println();
        System.out.println(
            "💡 Pattern matching funde verificação de tipo e cast em uma única expressão");
    }
}
```

```
🗡️ Hattori ataca com poder 8
```

```
💡 Pattern matching funde verificação de tipo e cast em uma única expressão.
```

# Java 17 | Sealed Classes

## A Hierarquia Protegida

Com *sealed*, você declara quais classes podem estender a sua, criando hierarquias fechadas e previsíveis. O compilador garante que nenhuma classe não autorizada herde sua *interface*, contribuindo para consistência do seu domínio.

```
SealedClasses.java

public class SealedClasses {
    // Tipos de técnicas permitidas no Bushidō - nunca expandir além disso
    public sealed interface TecnicaDeCombate permits Iaijutsu, Kendo, Jujutsu
    {
        String executar();
    }
    public static final class Iaijutsu implements TecnicaDeCombate {
        @Override
        public String executar() {
            return "🗡️ Saque rápido e corte";
        }
    }
    public static final class Kendo implements TecnicaDeCombate {
        @Override
        public String executar() {
            return "🏯 Combate com espada";
        }
    }
    public static final class Jujutsu implements TecnicaDeCombate {
        @Override
        public String executar() {
            return "🤼 Luta corpo a corpo";
        }
    }

    public static void main(String[] args) {
        List<TecnicaDeCombate> tecnicas = new ArrayList<>();
        tecnicas.add(new Iaijutsu());
        tecnicas.add(new Kendo());
        tecnicas.add(new Jujutsu());

        System.out.println("🗡️ Técnicas permitidas pelo Bushidō:");
        tecnicas.forEach(t -> System.out.println(" • " + t.executar()));
    }
}
```

```
🗡️ Técnicas permitidas pelo Bushidō:
• 🗡️ Saque rápido e corte
• 🏯 Combate com espada
• 🤼 Luta corpo a corpo
```



# **A FLUIDEZ DO AÇO JAVA 21**



# Java 21 | Visão Geral

## Arco V — A Fluidez do Aço

O Java 21, lançado em setembro de 2023, é a versão LTS que pode ser considerada a mais ambiciosa desde o Java 8. Ela traz *Virtual Threads*: uma mudança que redefine como o Java lida com concorrência, além de consolidar o *pattern matching* como ferramenta central da linguagem.

Com *Virtual Threads*, aplicações que fazem muitas chamadas de rede ou acesso a banco de dados podem escalar para milhões de conexões simultâneas sem a complexidade de programação reativa. O *pattern matching* no switch, por sua vez, transforma o antigo *switch* limitado em uma das estruturas mais poderosas da linguagem. E as *Sequenced Collections* unificam a forma de acessar elementos ordenados.

O Java 21 é a versão recomendada para novos projetos desde 2024. Se você está aprendendo Java agora, esta é a versão ideal para começar.

# Java 21 | Virtual Threads

## Mil Lâminas em Movimento

*Virtual threads* são *threads* leves gerenciadas pela JVM, não pelo Sistema Operacional. Você pode criar até milhões delas utilizando todo o potencial de memória e processamento. Ideais para aplicações *I/O-bound* como APIs REST e microsserviços.

```
VirtualThreads.java

public class VirtualThreads {

    public static void main(String[] args) {
        // Treinar samurais em paralelo sem sofrimento com virtual threads
        long inicio = System.currentTimeMillis();

        try (var dojo = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 1; i <= 10_000; i++) {
                int id = i;
                dojo.submit(() -> {
                    String resultado = treinarSamurai("Samurai-" + id);
                    System.out.println("✓ " + resultado);
                });
            }
        } // ThreadPool fecha automaticamente ao final do bloco

        long fim = System.currentTimeMillis();
        long duracao = fim - inicio;

        System.out.println();
        System.out.println("🏆 Tempo de execução: " + duracao + " ms");
        System.out.println("💡 Virtual threads: milhares de tasks leves gerenciadas pela JVM.}");

        private static String treinarSamurai(String nome) {
            try {
                Thread.sleep(10); // simula treino rápido
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            return nome + " completou o treino";
        }
    }
}
```

```
✓ Samurai-9994 completou o treino
✓ Samurai-9997 completou o treino
✓ Samurai-9996 completou o treino
✓ Samurai-9999 completou o treino
✓ Samurai-10000 completou o treino
✓ Samurai-9901 completou o treino
✓ Samurai-9897 completou o treino
✓ Samurai-9157 completou o treino

🏆 Tempo de execução: 365 ms
💡 Virtual threads: milhares de tasks leves gerenciadas pela JVM.
```



# Java 21 | Pattern Matching no switch

## O Corte Preciso

O `switch` agora suporta *pattern matching* com tipos, condições e *null*. Combinado com *sealed classes*, o compilador verifica se todos os casos foram cobertos, adeus cadeias de *if/else* com *instanceof*.

```
PatternMatchingSwitch.java

public class PatternMatchingSwitch {

    sealed interface Guerreiro permits SamuraiAtaque, SamuraiDefesa {
        String nome();
    }
    record SamuraiAtaque(String nome, int dano) implements Guerreiro {}
    record SamuraiDefesa(String nome, int armadura) implements Guerreiro {}

    public static void main(String[] args) {
        // Decidir próximo movimento de combate com elegância (pattern matching +
guards) Guerreiro atacante = new SamuraiAtaque("Musashi", 10);
        Guerreiro defensor = new SamuraiDefesa("Takeda", 8);

        System.out.println("Atacante: " + atacante.nome());
        System.out.println("Próximo movimento: " + proximoMovimento(atacante));

        System.out.println();
        System.out.println("Defensor: " + defensor.nome());
        System.out.println("Próximo movimento: " + proximoMovimento(defensor));
    }

    private static String proximoMovimento(Guerreiro guerreiro) {
        return switch (guerreiro) {
            case SamuraiAtaque s when s.dano() >= 8
                -> "Executar ataque supremo";
            case SamuraiAtaque s
                -> "Preparar ataque rápido";
            case SamuraiDefesa s when s.armadura() >= 7
                -> "Manter posição defensiva";
            case SamuraiDefesa s
                -> "Retrair para pontos fracos";
            default -> "Avaliar situação";
        };
    }
}
```

```
Atacante: Musashi
Próximo movimento: Executar ataque supremo

Defensor: Takeda
Próximo movimento: Manter posição defensiva
```

# Java 21 | Sequenced Collections

## A Ordem do Dojō

A interface *SequencedCollection* unifica o acesso a elementos ordenados com *getFirst()*, *getLast()* e *reversed()*. Funciona igual em *ArrayList*, *LinkedList* ou *TreeSet*, é uma API pequena com grande impacto.

```
SequencedCollections.java

public class SequencedCollections {

    public static void main(String[] args) {
        // Sequência de ataques em um duelo – ordem
        import java.util.SequencedCollection;
        SequencedCollection<String> ataques =
            new LinkedList<>();
        ataques.addAll(Arrays.asList(
            "Saque rápido",
            "Corte horizontal",
            "Contra-ataque",
            "Golpe final"
        ));

        System.out.println("Primeiro ataque: " +
            ataques.getFirst());
        System.out.println("Golpe final: " +
            ataques.getLast());
        System.out.println("Sequência reversa: " +
            ataques.reversed().stream().toList());
    }
}
```

```
Primeiro ataque: Saque rápido
Golpe final: Golpe final
Sequência reversa: [Golpe final, Contra-ataque, Corte horizontal, Saque rápido]
```



# **A LÂMINA AFIADA JAVA 25**



# Java 25 | Visão Geral

## Arco VI — A Lâmina Afiada

O Java 25 representa mais um passo no ciclo de evolução semestral que a plataforma adotou desde o Java 9. Nem toda versão traz recursos revolucionários e isso é intencional. Muitas versões focam em consolidar funcionalidades que estavam em *preview*, melhorar a performance da JVM e refinar a experiência do desenvolvedor.

Entre as melhorias desta versão, destacam-se o novo método *main*, a estabilização de APIs que vinham sendo testadas (como *Structured Concurrency* e *Scoped Values*), otimizações internas no coletor de lixo e aprimoramentos no compilador JIT. Para quem usa o Java no dia a dia, isso se traduz em aplicações mais rápidas e estáveis.

O ponto mais importante é entender a filosofia: o Java evolui de forma incremental e deliberada. Cada versão é um refinamento, não uma reinvenção.

*“O verdadeiro mestre refina, não reinventa.”*

# Java 25 | Método main simplificado

## Prossiga com Essência

No Java 25, iniciar um programa ficou ainda mais simples.

Agora, o método *main* pode ser definido sem *public static void* e sem parâmetros, tornando o código mais limpo e direto desde a primeira linha.

MainSimplificado.java

```
public class MainSimplificado {  
  
    void main() {  
        // Método main simplificado – sem static, sem String[] args  
        System.out.println("🏠 Dojo iniciado com método main simplificado.");  
        System.out.println("✂️ Sem necessidade de static ou parâmetros.");  
        System.out.println();  
        System.out.println("💡 Feature: Unnamed Classes and Instance Main  
Methods");  
        System.out.println("    Método 'void main()' é suficiente para executar.");  
    }  
}
```

🏠 Dojo iniciado com método main simplificado.

✂️ Sem necessidade de static ou parâmetros.

💡 Feature: Unnamed Classes and Instance Main Methods  
Método 'void main()' é suficiente para executar.

# Java 25 | Scoped Values

## Contexto Preciso, Missão Cumprida

*Scoped Values* permitem transmitir contexto dentro de um escopo de execução de forma segura, sem compartilhar estado global. É como passar uma mensagem sigilosa só para os métodos que realmente precisam dela.

```
ScopedValues.java

package dev.zancanela.javaversions.java25.scopedvalues;

public class ScopedValues {

    static final ScopedValue<String> MISSION = ScopedValue.newInstance();
    static final ScopedValue<String> DOJO = ScopedValue.newInstance();

    void main() {
        System.out.println("🗡 Mensagens do dojo:");

        ScopedValue.where(DOJO, "Kyoto Dojo")
            .where(MISSION, "Proteger o pergaminho")
            .run(ScopedValues::executarMissao);

        System.out.println();

        ScopedValue.where(DOJO, "Tokyo Dojo")
            .where(MISSION, "Recuperar a espada sagrada")
            .run(ScopedValues::executarMissao);

        System.out.println();
        System.out.println("💡 Scoped Values: contexto por escopo, sem ThreadLocal.");

        private static void executarMissao() {
            System.out.println("Dojo: " + DOJO.get());
            System.out.println("Missão: " + MISSION.get());
            System.out.println("→ Ação executada com contexto isolado");
        }
    }
}
```

```
🗡 Mensagens do dojo:
Dojo: Kyoto Dojo
Missão: Proteger o pergaminho
→ Ação executada com contexto isolado

Dojo: Tokyo Dojo
Missão: Recuperar a espada sagrada
→ Ação executada com contexto isolado

💡 Scoped Values: contexto por escopo, sem ThreadLocal.
```

# Java 25 | Flexible Constructors

## A Inicialização Harmoniosa

No Java 25, objetos podem ser criados de várias formas, sem precisar de múltiplos construtores ou fábricas. Isso torna o código mais limpo e evita duplicação de lógica. O resultado é um design mais claro e alinhado ao domínio do problema.

```
FlexibleConstructors.java

public class FlexibleConstructors {

    static class Guerreiro {
        String nome;
        int nivel;

        Guerreiro(String nome, int nivel) {
            this.nome = nome;
            this.nivel = nivel;
        }
    }

    static class Samurai extends Guerreiro {
        String escola;

        // Flexible Constructor Bodies: validação ANTES de super()
        Samurai(String nome, int nivel, String escola) {
            if (nivel < 0 || nivel > 10)
                throw new IllegalArgumentException(
                    "Não foi possível inscrever \"" +
                    nome +
                    "\" - 0 nível deve estar entre 0 e 10");
            super(nome, nivel);
            this.escola = escola;
        }
    }

    void main() {
        try {
            Samurai musashi = new Samurai("Musashi", 10, "Niten'ichi");
            System.out.println("✓ " + musashi.nome + " (nível " + musashi.nivel + ")");
        } catch (IllegalArgumentException e) {
            System.out.println("✗ " + e.getMessage());
        }

        try {
            new Samurai("Yoshi", 15, "Tenshin");
        } catch (IllegalArgumentException e) {
            System.out.println("✗ " + e.getMessage());
        }
    }
}
```

```
✓ Musashi (nível 10)
```

```
✗ Não foi possível inscrever "Yoshi" - 0 nível deve estar entre 0 e 10
```

# O CAMINHO DO JAVA BUSHIDŌ





# Evoluir Sem Romper

## Arco Final — O Caminho do Java Bushidō

Com tantas versões disponíveis, uma dúvida comum entre desenvolvedores iniciantes é: qual versão devo usar? A resposta é simples: em produção, priorize sempre versões *LTS* (*Long-Term Support*). Atualmente, as versões LTS mais relevantes são o Java 17, 21 e 25. Elas recebem atualizações de segurança e correções por anos, o que é essencial para sistemas em produção.

Versões intermediárias (como 20, 22, 23, 24) são ótimas para experimentação e aprendizado. Elas permitem testar novos recursos antes que sejam consolidados nas versões LTS. Use-as em projetos pessoais, estudos e provas de conceito, mas evite colocá-las em ambientes críticos.

Planeje upgrades de versão como parte da arquitetura do projeto, não como uma emergência. Inclua atualizações de Java no backlog e aproveite as notas de release para entender o que mudou. Esse hábito é o que diferencia equipes maduras de equipes que ficam presas em versões antigas por falta de planejamento.

# Epílogo — Honra no Código

## O caminho continua

Chegamos ao final desta jornada pelo universo do Java moderno. Ao longo deste ebook, percorremos a evolução de uma das linguagens mais relevantes do mundo.

O que diferencia um bom desenvolvedor não é apenas conhecer a sintaxe mais recente, mas entender o propósito por trás de cada mudança. Código limpo não é só questão de estética, é também disciplina. Evolução não é só sobre novos recursos, é também constância. E precisão técnica não é perfeccionismo, é impacto real no produto e na equipe.

O caminho do Java Bushidō é contínuo. Cada versão traz novas oportunidades de aprendizado, e cada linha de código que você gera é uma chance de praticar o que aprendeu aqui. Continue estudando, continue codando, e leve com você a mentalidade de que evolução técnica é um compromisso diário.

*“Obrigado por seguir o caminho do Java Bushidō.”*